A DEEP LEARNING MINI-PROJECT

on

8-Puzzle Solver

Comparing Q-Learning, Deep Q-Network (DQN) and A* Search with an Interactive Web Demo

Submitted towards the
Fulfilment of the Requirements of
Bachelor of Technology in

Computer Engineering

By

Name	Roll No.	Seat No.
Meghraj Nilesh Bhavsar	1	B231301011
Sahil Nandkishor Patil	2	B231301095
Vaibhavi Kamalakar Dhokale	3	B231301028
Areen Vikram Shinde	4	B231301135

Under the guidance of

Prof. Shweta Jadhav

Department of Computer Engineering

K. K. WAGH EDUCATION SOCIETY'S
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
Hirabai Haridas Vidyanagari, Amrutdham, Panchavati, Nashik, Maharashtra 422003
(An Autonomous Institute affiliated to Savitribai Phule Pune University)

A. Y. 2026-2027 Sem VII

Abstract

The 8-puzzle is a 3×3 sliding-tile puzzle in which the tiles 1 to 8 must be put in order by sliding them into the single empty cell. It has only 181,440 reachable arrangements, so a search algorithm can solve it optimally, which makes it a good test bed for comparing learning-based solvers against a known correct answer.

This project builds and compares three solvers: **A*** search with the Manhattan-distance heuristic (the optimal baseline), **tabular Q-learning**, and a **Deep Q-Network (DQN)** in two versions. The learners are trained with a shared reward (+10 at the goal, −0.1 per move and a small shaping bonus), a curriculum that increases the scramble depth as the agent improves, and resumable, seed-controlled, CPU-only training. All methods are tested on a fixed set of 1,800 boards (200 for each scramble depth from 1 to 30) that are never used as training start boards.

On this test set A* solves 100% of the boards optimally. Tabular Q-learning solves 96.7%, the first DQN (81-128-128-4 network) solves 79.3%, and the improved DQN v2 (81-256-256-4 network with Double DQN, soft target updates, learning-rate decay and validation-based model selection) solves 88.4%. Learned solvers are nearly optimal on boards they solve, but they fail by looping on deep scrambles, and failures are reported honestly with no fallback to A*. A Flask web application lets a user upload a picture, scramble it, watch each solver, compare all four side by side, or play the puzzle with hints.

Keywords: 8-puzzle, Reinforcement Learning, Q-learning, Deep Q-Network, Double DQN, A* search, Curriculum Learning, Reward Shaping, Flask.

Contents

Abstract . . . . . **i**
1 Introduction **1**
1.1 Background 1
1.2 Motivation 1
1.3 Objectives 1
1.4 Scope 2
2 Problem Statement **3**
2.1 Challenges 3
3 Literature Survey **4**
4 Puzzle Environment and Test Data **5**
4.1 Environment Details 5
4.2 Solvability and Board Generation 5
4.3 Fixed Test Set 6
4.4 Training and Validation Boards 6
5 State Representation and Reward Design **7**
5.1 State Representation 7
5.2 Illegal Action Masking 7
5.3 Reward Design 8
5.4 Curriculum Learning 8
6 Model Architecture **9**
6.1 A* Search (Baseline) 9
6.2 Tabular Q-Learning 9
6.3 Deep Q-Network (DQN v1) 9
6.4 DQN v2: The Second Network 10
6.5 Greedy Solver and Failure Reasons 11
7 Model Training **12**
7.1 Training Pipeline 12
7.2 Training Parameters 12
7.3 Loss Function and Optimizer 13
7.4 Training Steps 13
7.5 Checkpoints, Seeds and Snapshots 13
7.6 Training Time and Number of Iterations 14
7.7 Tuning Experience 14
8 System Implementation **15**
8.1 System Architecture 15
8.2 Picture Upload and Tile Rendering 15
8.3 Server API 16
8.4 Using the Application 16
8.5 Training Results Panel and Mobile Layout 21
8.6 Technologies Used 24
8.7 Running the Project 24
9 Results **25**
9.1 Evaluation Metrics 25
9.2 Overall Performance 25
9.3 Performance by Scramble Depth 26

9.4 Solution Quality . . . 27
9.5 Learning Curves and Reward . . . 27
9.6 Improvement During Training (Snapshots) . . . 28
9.7 Generalization of the Q-Table . . . 29
9.8 Failure Analysis . . . 29
9.9 Example Solution . . . 29
9.10 An Extra Experiment: Starting Q-Learning from Random Boards . . . 30
10 Observations . . . 31
10.1 What Worked Well . . . 31
10.2 What Did Not Work Well . . . 31
10.3 Why the DQN Does Worse than the Table . . . 31
10.4 Effect of the Design Decisions . . . 32
10.5 Limits of the Results . . . 32
10.6 Summary . . . 32
11 Conclusion and Future Scope . . . 33
11.1 Conclusion . . . 33
11.2 Future Scope . . . 33
References . . . 34
Appendix A Project Structure . . . 35
Disclaimer . . . 36

List of Figures

5.1 One-hot encoding of a board into 81 network inputs 7
6.1 The two networks: DQN v1 (81-128-128-4) and DQN v2 (81-256-256-4) 10
7.1 Training pipeline 12
8.1 System architecture of the web application 15
8.2 Home page of the web application with the default picture 17
8.3 A user-uploaded picture, cropped to a square and scrambled with 20 moves 17
8.4 Watch AI mode: A* animating a solution 18
8.5 A* has solved the puzzle (optimal moves and time are shown) 18
8.6 A failure is shown honestly: DQN v1 looped, and the board it reached is displayed 19
8.7 DQN v2 solving a board 19
8.8 Compare mode: four solvers on the same depth-25 board 20
8.9 Play mode with a hint (highlighted tile) 21
8.10 Training-results panel inside the web page 22
8.11 The page on a phone-sized screen (390 px wide) 23
9.1 Solve rate by scramble depth for the four methods 26
9.2 Solve rate on the test boards during training (logarithmic episode axis) 27
9.3 Average reward per training episode 28
9.4 Solve rate by depth at about 0%, 25%, 50% and 100% of training 28
9.5 Q-learning: test boards whose start state is in the table (seen) or not (unseen) 29
9.6 Frames of an optimal A* solution for a depth-20 test board 29

List of Tables

2.1 Challenges and how they are handled . . . 3
3.1 Summary of related work . . . 4
4.1 Environment specifications . . . 5
4.2 Fixed test set (200 boards per depth) . . . 6
6.1 Layer-wise details of DQN v1 . . . 10
6.2 Layer-wise details of DQN v2 . . . 11
7.1 Training parameters . . . 12
7.2 Number of iterations and training time (one CPU thread) . . . 14
7.3 Main experiments while tuning the DQN (test solve rate) . . . 14
8.1 Routes of the Flask application . . . 16
8.2 Technologies used . . . 24
9.1 Overall performance on the 1,800 test boards . . . 25
9.2 Solve rate by scramble depth . . . 26
9.3 Extra moves compared with A* on the boards each method solved . . . 27
9.4 Why the learned solvers failed (1,800 test boards) . . . 29
10.1 Summary of design decisions . . . 32
10.2 Summary of the three approaches . . . 32
11.1 Future scope . . . 33

Chapter 1
Introduction

1.1 Background

The sliding-tile puzzle is one of the oldest puzzles used in artificial intelligence. In the 8-puzzle a 3×3 frame holds eight numbered tiles and one empty cell. A move slides a neighbouring tile into the empty cell, and the aim is to reach the arrangement 1, 2, 3, 4, 5, 6, 7, 8 followed by the empty cell. There are 9! = 362,880 ways to place the nine objects, but exactly half of them can never be solved; this was shown by Johnson and Story in 1879 using a parity argument [9]. The remaining 181,440 arrangements are all connected, and the hardest of them needs 31 moves [16].

Because the space is small, the puzzle can be solved exactly by graph search (A* [3], IDA* [7]). Because the correct answer is available, it is also a clean place to study *learning* methods: we can measure not only whether a learned agent succeeds, but also how far its solutions are from the optimum. The same ideas are used on much larger problems, for example the Rubik's Cube solver DeepCubeA [8].

1.2 Motivation

In our Deep Learning course we studied reinforcement learning, neural networks, loss functions, optimizers and model evaluation. We wanted a project in which all of these are applied to one problem and in which the result can be checked against a trusted baseline. The 8-puzzle gives us that: tabular Q-learning [1] and Deep Q-Networks [2] can be compared with A* on the same boards. We also wanted the work to be easy to show, so we built a web page where anybody can scramble a picture and watch the three methods solve it.

1.3 Objectives

1. To build a puzzle environment that only produces solvable boards and a correct A* solver to act as the optimal baseline.
2. To create a fixed test set of 1,800 boards with known optimal lengths that is never used for training.
3. To train a tabular Q-learning agent and a Deep Q-Network (DQN) with a shared reward and a depth curriculum, on a laptop CPU, with resumable training.
4. To design a second, improved network (DQN v2) and explain which design decisions helped and which did not.
5. To evaluate all methods using solve rate, average moves, optimality gap and solve time, per scramble depth.

6. To build a Flask web application to watch, compare and play, and to report solver failures honestly.

1.4 Scope

The project is an educational prototype. It solves the 3×3 puzzle only (the code takes the board size as a parameter, but all experiments use 3×3). All results come from one training run per method with seed 0, so small differences between methods should not be over-interpreted. The learned solvers carry no guarantee of success; when they fail, the failure is shown instead of being hidden by calling A*.

Chapter 2
Problem Statement

"To develop reinforcement-learning based solvers (tabular Q-learning and Deep Q-Networks) for the 8-puzzle, to compare them with the optimal A algorithm on a fixed set of test boards, and to present the results through an interactive web application in which a user can scramble a picture, watch the solvers, and play the puzzle."*

2.1 Challenges

Table 2.1: Challenges and how they are handled

Challenge	How the project handles it
Half of all arrangements are unsolvable	Boards are made by playing random legal moves backwards from the goal; a parity test is also provided
Reward only at the goal (sparse reward)	Curriculum: start with 1–3 move scrambles and make them harder as the solve rate improves; Manhattan-distance shaping reward
Learned policies can walk in circles	Greedy solver has a step cap and loop detection and reports the failure reason
Evaluation must be fair	1,800 fixed test boards, never used as training start boards; separate validation boards for model selection
Deep Q-learning is unstable	Target network, Huber loss, gradient clipping, then Double DQN, soft target updates and learning-rate decay
Training must fit a laptop	CPU only, small networks, checkpoints every few thousand episodes, safe to stop and resume
Results must be understandable	Plots, GIFs and a web demo with an honest "failed to solve" state

Chapter 3
Literature Survey

Search methods came first. Hart, Nilsson and Raphael [3] introduced A*, which expands the node with the smallest value of f = g + h, where g is the cost so far and h is an estimate of the remaining cost. If h never over-estimates, the first solution found is optimal. Korf [7] added iterative deepening (IDA*), and Reinefeld [16] solved the 8-puzzle completely, finding that the longest optimal solution has 31 moves.

Learning methods followed. Watkins and Dayan [1] proved that Q-learning converges to the optimal action values if every state and action is tried often enough. Mnih et al. [2] showed that a neural network trained with experience replay [12] and a separate target network can learn control from raw inputs (DQN). van Hasselt et al. [4] noticed that DQN over-estimates values and proposed Double DQN. Ng, Harada and Russell [5] proved that a shaping reward of the form gamma·phi(s') − phi(s) does not change the optimal policy, and Bengio et al. [6] showed that training on easy examples first (curriculum learning) can speed up learning. Agostinelli et al. [8] combined deep reinforcement learning with search to solve the Rubik's Cube.

Table 3.1: Summary of related work

Paper	Method	Finding
Hart et al., 1968 [3]	A*	Heuristic search is optimal when the heuristic never over-estimates
Korf, 1985 [7]	IDA*	Optimal search with very little memory
Watkins & Dayan, 1992 [1]	Q-learning	Converges to optimal values with enough exploration
Lin, 1992 [12]	Experience replay	Re-using stored experience makes learning more efficient
Ng et al., 1999 [5]	Reward shaping	Potential-based shaping keeps the optimal policy
Bengio et al., 2009 [6]	Curriculum learning	Easy-to-hard training can help
Mnih et al., 2015 [2]	DQN	Neural Q-function with replay and target network
van Hasselt et al., 2016 [4]	Double DQN	Reduces over-estimation in the DQN target
Agostinelli et al., 2019 [8]	DeepCubeA	Deep RL plus search solves the Rubik's Cube

Most of this work uses large problems and large computing budgets. Our project applies the same ideas to a small puzzle with a known optimal answer, so that every design choice can be tested and measured on a laptop, and adds a web demo.

Chapter 4
Puzzle Environment and Test Data

This project does not use an image dataset. The “data” are puzzle boards that we generate ourselves, so this chapter describes the environment and how the training and test boards are created.

4.1 Environment Details

Table 4.1: Environment specifications

Item	Value
Board	3×3 grid, tiles 1–8, blank = 0
State	tuple of 9 integers, row by row; goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
Actions	0 = up, 1 = down, 2 = left, 3 = right (they move the blank); moves off the board are illegal
All arrangements	9! = 362,880
Solvable arrangements	181,440 (exactly half)
Longest optimal solution	31 moves [16]
Board size	a parameter (shape = (rows, cols)); we tested the code on 2×3 first

4.2 Solvability and Board Generation

An arrangement is solvable only if its number of *inversions* is even. An inversion is a pair of tiles (ignoring the blank) that appear in the wrong order when the board is read row by row. For example (2, 1, 3, 4, 5, 6, 7, 8, 0) has one inversion and can never be solved. The function `is_solvable` implements this test, and a unit test checks it against a brute-force search on the 2×3 board.

Random permutations are never used to create boards, because half of them are unsolvable. Instead a board is created by **reverse scrambling**: start at the goal and play N random legal moves, never undoing the previous move. N is called the *scramble depth*. Every such board is solvable by construction. A deeper scramble usually means a harder board, but not always: a random walk can wander back, so the optimal length is often shorter than the depth (Table 4.2).

4.3 Fixed Test Set

The test set was created once with a fixed seed (12345). It has 200 boards for each of nine scramble depths, 1,800 boards in total. The A* optimal length of every board is stored with it. Table 4.2 summarizes it. At small depths there are very few different boards (depth 1 has only 2), so the test set contains repeats there; this is a property of the puzzle, not a mistake.

Table 4.2: Fixed test set (200 boards per depth)

Scramble depth	Boards	Distinct boards	Avg. optimal length	Max. optimal length
1	200	2	1.00	1
3	200	8	3.00	3
5	200	20	5.00	5
8	200	82	7.90	8
12	200	175	11.41	12
16	200	194	14.55	16
20	200	200	17.48	20
25	200	200	19.23	25
30	200	199	20.05	28

4.4 Training and Validation Boards

- **Training boards:** created on the fly by reverse scrambling with the depth chosen by the curriculum. A board that is in the test set is never used as a training start board; where all boards of a shallow depth are in the test set, a slightly deeper scramble is used instead. (The agent can of course pass through test-board states while it plays an episode, since those are ordinary board positions.)
- **Validation boards:** for DQN v2 only, 240 extra boards (40 each at depths 8, 12, 16, 20, 25, 30; seed 777) that are neither test boards nor training start boards. They are used to choose the best saved network during training, so that the test set is used only for the final comparison.

Chapter 5
State Representation and Reward Design

Before learning, the board, the actions and the reward must be put in a form the learners can use. Both Q-learning and DQN use the same environment, reward and curriculum, so any difference in results comes from the learning method itself.

5.1 State Representation

- **Q-learning:** the state tuple is used directly as the key of a Python dictionary.
- **DQN:** the board is converted to a **one-hot** vector of 81 numbers: for each of the 9 cells there are 9 inputs (blank, 1, ..., 8), and the input that matches the tile in that cell is 1. Exactly nine of the 81 inputs are 1 (Figure 5.1). One-hot coding is used because tile numbers are labels, not quantities: tile 8 is not “twice” tile 4.

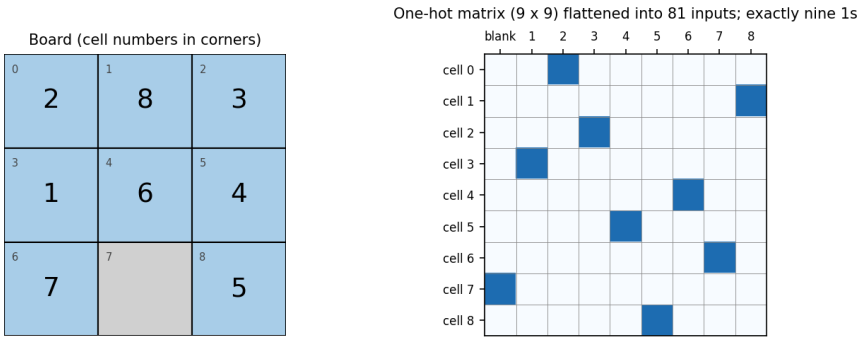


Figure 5.1: One-hot encoding of a board into 81 network inputs

5.2 Illegal Action Masking

Some actions are impossible (for example the blank in the top row cannot move up). The Q-values of impossible actions are set to minus infinity, so that the maximum and the argmax operations can never choose them, and the agent does not waste learning on them. In the Q-table these entries are stored as $-\infty$ at creation; in the DQN they are masked after the network output.

5.3 Reward Design

The agent gets +10 for reaching the goal and −0.1 for every move, so shorter solutions earn more. Only these two rewards would be very sparse for deep scrambles, so a small **shaping** bonus is added. With h(s) the Manhattan distance of board s (the sum over tiles of their row distance plus column distance to the goal cell), the reward for a move from s to s' is given in Equation 5.1.

$$r_t = -0.1 + 0.3\,[h(s_t) - \gamma\, h(s_{t+1})]\ (+10 \text{ at the goal}) \qquad (5.1)$$

A move that lowers the Manhattan distance gives a small positive bonus and a move that raises it a small penalty. This form is called *potential-based* shaping and, as proved in [5], it does not change which policy is optimal; it only speeds up learning. The discount factor is gamma = 0.99.

5.4 Curriculum Learning

A random board is solved by chance very rarely, so an agent that starts on deep scrambles would almost never see the +10 reward. The curriculum therefore starts with scramble depths 1 to 3. Every training episode draws a depth uniformly from 1 to *max_depth*. The solve rate over the last 300 training episodes is tracked; when it reaches 80%, max_depth is increased by one and the window is restarted. The maximum is 32. An episode is stopped after max(12, 4 × depth) moves. Because depths are drawn from the whole range 1 to max_depth, easy boards keep being practised and are not forgotten.

Chapter 6
Model Architecture

6.1 A* Search (Baseline)

A* keeps a priority queue of boards ordered by f(n) = g(n) + h(n) (Equation 6.1), where g is the number of moves made so far and h is the Manhattan distance. The Manhattan distance never over-estimates the true number of moves left, so the first time the goal is taken from the queue, the path is optimal. A node limit stops the search if it runs too long. A* does not learn; it is used to create the optimal lengths in the test set, as the baseline in all comparisons, and for hints in the web application.

$$f(n) = g(n) + h(n) \qquad h(n) = \sum_{\text{tiles}} \left(|r - r_{goal}| + |c - c_{goal}| \right) \tag{6.1}$$

6.2 Tabular Q-Learning

The Q-table is a Python dictionary that maps every board seen so far to an array of four numbers, one Q-value per action. After each move the value of the move taken is updated with Equation 6.2, where alpha = 0.3 is the learning rate. At the goal there is no next board, so the target is just the reward. Moves are chosen *epsilon-greedily*: with probability epsilon a random legal move is played (exploration), otherwise the best known move (exploitation). Epsilon falls from 1.0 to 0.05 during the first 60% of training. Ties between equal Q-values are broken randomly during training.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{6.2}$$

After 600,000 episodes the table held 164,443 boards, which is 91% of all 181,440 solvable boards.

6.3 Deep Q-Network (DQN v1)

A table cannot say anything about a board it has never visited, and for bigger puzzles it would not fit in memory. A DQN replaces the table with a neural network that takes the board and outputs the four Q-values. Version 1 is a fully connected network with two hidden layers of 128 neurons (Figure 6.1, Table 6.1). Training uses these ideas from [2]:

- **Experience replay:** every transition (board, action, reward, next board) is stored in a buffer of 100,000 entries and training uses random batches of 128 from it. This breaks the strong correlation between consecutive moves.
- **Target network:** the training target is computed with a second, slowly changing copy of the network (Equation 6.3), which keeps the target from moving at every step.

- **Huber loss** (Equation 6.4) is quadratic for small errors and linear for large ones, so rare large errors do not give huge gradients.
- **Adam optimizer** [11] with learning rate 3×10^{-4} and **gradient clipping** at norm 10.

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') \qquad L = \text{Huber}\,(Q_{\text{online}}(s, a) - y) \tag{6.3}$$

$$L_\delta(x) = \tfrac{1}{2}x^2 \;\; (|x| \le 1), \qquad L_\delta(x) = |x| - \tfrac{1}{2} \;\; (|x| > 1) \tag{6.4}$$

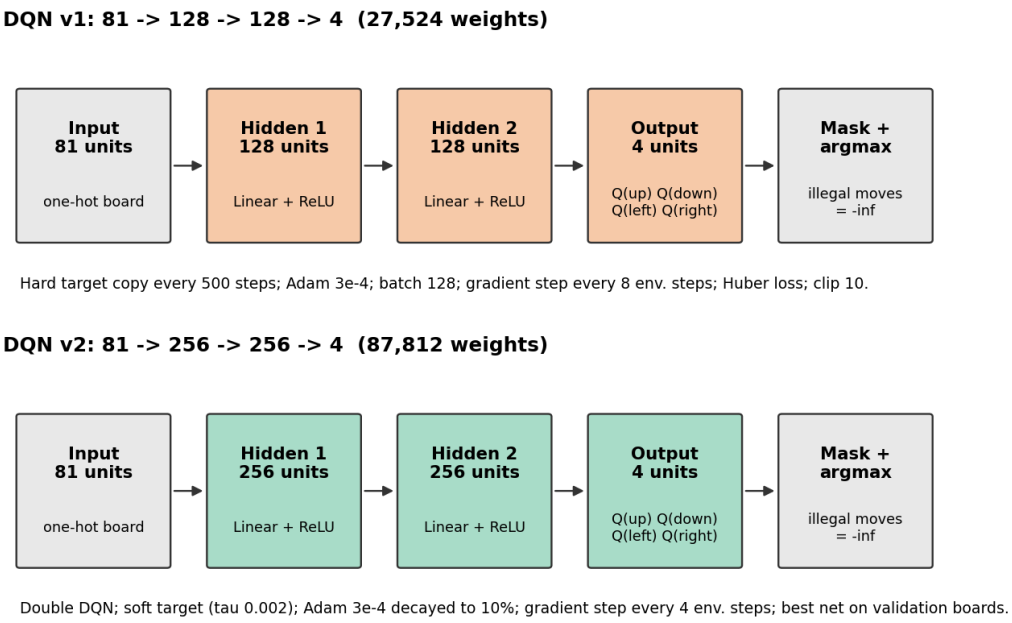


Figure 6.1: The two networks: DQN v1 (81-128-128-4) and DQN v2 (81-256-256-4)

Table 6.1: Layer-wise details of DQN v1

Layer	Output	Parameters
Input (one-hot board)	81	0
Linear 81 → 128 + ReLU	128	10,496
Linear 128 → 128 + ReLU	128	16,512
Linear 128 → 4 (Q-values)	4	516
Total		**27,524**

6.4 DQN v2: The Second Network

DQN v1 solved most shallow boards but failed on deep ones, almost always by *looping* between two neighbouring boards (Chapter 9). DQN v2 keeps the same input, reward, curriculum, replay buffer and loss, and changes the learning method and the size of the network:

- **Wider layers:** 256 neurons per hidden layer instead of 128 (87,812 parameters, Table 6.2).

- **Double DQN** [4]: the online network *chooses* the best next action and the target network *evaluates* it (Equation 6.5). The plain DQN target takes the maximum over noisy estimates, which is biased upwards.
- **Soft target update:** instead of copying the weights every 500 steps, the target moves a little towards the online network after every step (Equation 6.6, tau = 0.002). This gave smoother and more stable learning.
- **Learning-rate decay:** the Adam learning rate falls linearly from 3×10^{-4} to 10% of that value, so that late updates are small corrections.
- **Validation-based model selection:** the network is evaluated on 240 validation boards at every log point and the best one is kept.

$$y = r + \gamma\, Q_{\text{target}}\left(s',\ \arg\max_{a'} Q_{\text{online}}(s', a')\right) \tag{6.5}$$

$$\theta_{\text{target}} \leftarrow (1-\tau)\,\theta_{\text{target}} + \tau\,\theta_{\text{online}} \qquad \tau = 0.002 \tag{6.6}$$

Table 6.2: Layer-wise details of DQN v2

Layer	Output	Parameters
Input (one-hot board)	81	0
Linear 81 → 256 + ReLU	256	20,992
Linear 256 → 256 + ReLU	256	65,792
Linear 256 → 4 (Q-values)	4	1,028
Total		**87,812**

6.5 Greedy Solver and Failure Reasons

After training, a board is solved by repeatedly playing the legal move with the highest Q-value. The same routine is used for the Q-table and both networks. It stops as soon as one of the following happens: the board is **solved**; **step_cap** – 100 moves were used; **loop** – a board is visited for the third time; **unseen_state** – (Q-table only) the agent has reached a board that is not in its table. If the solver fails, it says so. There is **no fallback to A***, because that would hide the weakness of the learned policy.

Chapter 7
Model Training

7.1 Training Pipeline

Figure 7.1 shows one training loop. The same loop is used for Q-learning and DQN; only the “learn” step differs (a table update or a gradient step on a batch from the replay buffer).

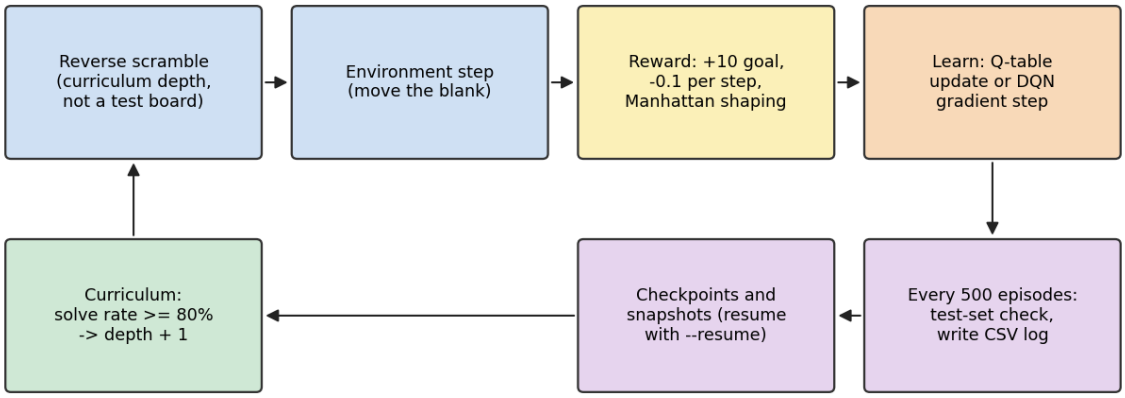


Figure 7.1: Training pipeline

7.2 Training Parameters

Table 7.1: Training parameters

Parameter	Q-learning	DQN v1	DQN v2
Episodes	600,000	80,000	60,000
Hidden layers	(table)	128, 128	256, 256
Learning rate	alpha = 0.3	Adam 3e-4	Adam 3e-4 → 3e-5 (linear decay)
Discount gamma	0.99	0.99	0.99
Epsilon	1.0 → 0.05 over first 60% of episodes	same	same
Batch / replay buffer	–	128 / 100,000	128 / 100,000
Learning starts after	–	2,000 transitions	2,000 transitions
Gradient step every	–	8 env. steps	4 env. steps
Target network	–	hard copy every 500 steps	soft update, tau = 0.002 every step
Target value	max over actions	max over actions	Double DQN

Parameter	Q-learning	DQN v1	DQN v2
Loss / clipping	–	Huber / norm 10	Huber / norm 10
Curriculum	start 3, cap 32, window 300, threshold 80%	same	same
Step cap per episode	max(12, 4 × depth)	same	same
Best-model selection	–	none (final network)	240 validation boards
Seed / device	0 / CPU	0 / CPU, 1 thread	0 / CPU, 1 thread

7.3 Loss Function and Optimizer

The DQNs minimise the Huber loss between the predicted Q-value of the move that was played and the target of Equation 6.3 (v1) or 6.5 (v2). Gradients are clipped to norm 10 and applied with the Adam optimizer. Q-learning has no loss function; its update rule (Equation 6.2) moves each table entry directly towards its target.

7.4 Training Steps

1. Pick a scramble depth from the curriculum range and create a board by reverse scrambling (not a test or validation board).
2. Play up to max(12, 4 × depth) moves, choosing each move epsilon-greedily among the legal moves.
3. After each move compute the reward and update the Q-table, or store the transition and take a gradient step on a random batch (every 8 or 4 steps).
4. At the end of the episode tell the curriculum whether the goal was reached; deepen the curriculum when the solve rate is at least 80%.
5. Every 500 episodes solve all test boards greedily and write one row to the CSV log (solve rate, average moves, reward, loss, elapsed time). DQN v2 also scores the validation boards and keeps the best network.
6. Save checkpoints periodically and snapshots at about 0%, 25%, 50% and 100% of the training.

7.5 Checkpoints, Seeds and Snapshots

Every training script can be stopped at any time and continued with the `--resume` flag. A checkpoint stores the table or networks, the optimizer, the replay buffer, the curriculum state, the random-number generator states and the log so far. Files are written to a temporary name and then renamed, so an interruption can never leave a half-written file. All random numbers come from a seed that can be set with `--seed` (default 0). The snapshots at 0%, 25%, 50% and 100% of training are kept so that early and late behaviour can be compared; the “0%” snapshot is taken after the first 500 episodes, because an untrained agent can do nothing.

7.6 Training Time and Number of Iterations

Table 7.2: Number of iterations and training time (one CPU thread)

Run	Episodes	Env. steps	Gradient steps	Time
Q-learning	600,000	about 8–10 million (estimated)	–	2.9 min
DQN v1	80,000	2,159,009	269,627	10.2 min
DQN v2	60,000	1,492,316	372,580	21.8 min

Running a short “quick” mode for each training script (about one minute) is available to test the whole pipeline before a long run.

7.7 Tuning Experience

The first DQN run (learning rate 1e-3, target copy every 1,000 steps) diverged: the test solve rate climbed to about 58% and then collapsed to 5% while the loss kept growing. Table 7.3 lists the main experiments that followed. Most runs were stopped early when it was clear they were not improving, and the results are read from the test-set log, so the DQN numbers in Chapter 9 are slightly optimistic (Chapter 10).

Table 7.3: Main experiments while tuning the DQN (test solve rate)

Run	Change	Result
v1-0	lr 1e-3, hard copy 1,000, gradient step every 4	peaked near 58%, collapsed to 5%
v1-A	lr 3e-4, hard copy 2,000	65% at 29k episodes (stopped)
v1-B	lr 1e-4, hard copy 4,000	70% at 34k episodes (stopped)
v1-C (chosen)	**lr 3e-4, hard copy 500, step every 8**	**70% at 40k; 79.3% at 80k**
v2-a	n-step 5, Double DQN, 3×256	62.8% at 40k
v2-b	n-step 10, Double DQN, 3×256	55.1% at 40k
v2-c	n-step 5, Double DQN, 2×128	64.1% at 40k
v2-d	n-step 3, Double, 3×256, step every 2, eps_end 0.02	54.6% at 16k (stopped)
v2-e	Double, 2×128, step every 2, eps_end 0.02	collapsed to 14% at 24k (stopped)
v2-f	n-step 3, Double, 3×256, batch 256, lr 2e-4	55.0% at 40k
v2-g	soft target (tau 0.002), n-step 3, 3×256	57.5% at 20k (stopped)
v2-h	soft target, Double, 2×128	peak 74% at 20k, 67% at 32k (stopped)
v2-i	v2-h + lr decay + validation selection, 60k episodes	test 88.6% (best validation 80.4%)
v2-j (chosen)	**v2-i with 2×256**	**test 88.4% (best validation 82.9%)**

The two runs v2-i and v2-j were compared on the *validation* boards and v2-j was selected. The experiments show that the gain of DQN v2 came mainly from soft target updates, Double DQN and learning-rate decay; n-step returns and deeper networks did not help in our setting, and doing more gradient steps per environment step made training unstable.

Chapter 8
System Implementation

8.1 System Architecture

The trained models are used inside a small web application written with Flask. Figure 8.1 shows how the parts are connected. The browser sends a board to the server; the server checks it, runs the chosen solver and returns the list of moves, and the browser animates them. The models are loaded once when the server starts. If a model file is missing, the page shows a clear message and the other solvers keep working.

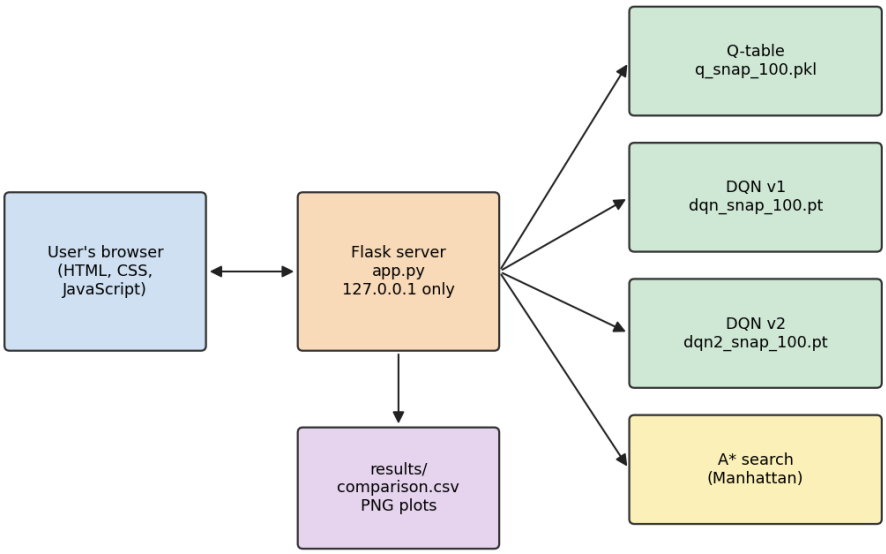


Figure 8.1: System architecture of the web application

8.2 Picture Upload and Tile Rendering

The user can upload a PNG, JPEG or WEBP picture of at most 5 MB. The type and size are checked in the browser and again on the server, where the picture is decoded in memory with Pillow, centre-cropped to a square and shrunk to at most 600 pixels. The server never writes the picture to disk. The browser then draws each tile with a CSS background image and a background-position offset, so no tile images have to be stored anywhere. The bottom-right cell is the blank. A built-in picture is used until the user uploads one.

8.3 Server API

Table 8.1: Routes of the Flask application

Route	Purpose
GET /	The page
GET /api/status	Which solvers are available (and why not)
POST /api/upload	Validate, crop and return the uploaded picture
POST /api/scramble	Make a solvable board for a depth between 3 and 30
POST /api/solve	Solve a board with qlearning, dqn, dqn2 or astar; returns moves, solved, number of moves, time, reason, final board
POST /api/compare	Run all four solvers on the same board
POST /api/hint	Next move and remaining optimal length from A*
GET /api/results	Overall rows of comparison.csv and available plots

Every route that receives a board checks that it is a list of nine integers that is a permutation of 0–8 and is solvable; errors are returned as JSON messages with a proper status code. The server listens on 127.0.0.1 only and debug mode is off by default.

8.4 Using the Application

Figures 8.2 and 8.3 show the page, first with the default picture and then after a picture has been uploaded and scrambled. The page has three steps: choose a picture, choose the difficulty (scramble depth 3 to 30) and press Scramble, then pick a mode.

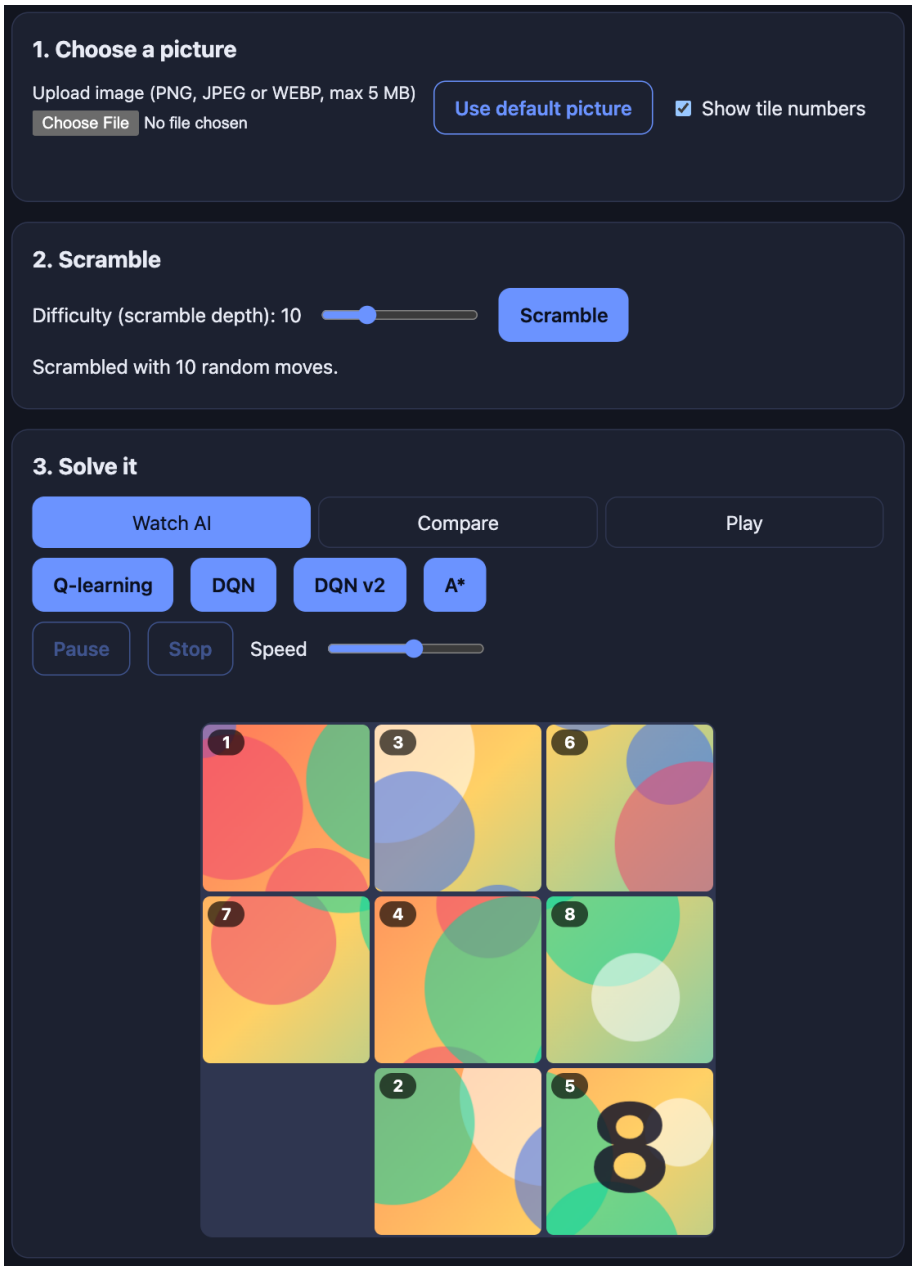


Figure 8.2: Home page of the web application with the default picture

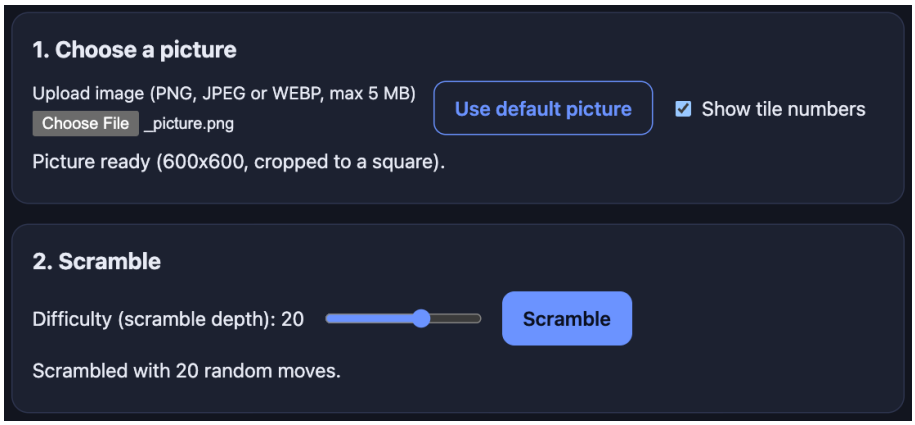


Figure 8.3: A user-uploaded picture, cropped to a square and scrambled with 20 moves

Watch AI

In this mode one solver is run and its moves are animated; the speed can be changed and the animation can be paused or stopped. Figure 8.4 shows A* in the middle of a solution and Figure 8.5 the finished puzzle. Figure 8.6 shows a result that is reported honestly: DQN v1 got stuck repeating boards, so the page says “Failed to solve” and shows the board it reached. Figure 8.7 shows DQN v2 solving a board.

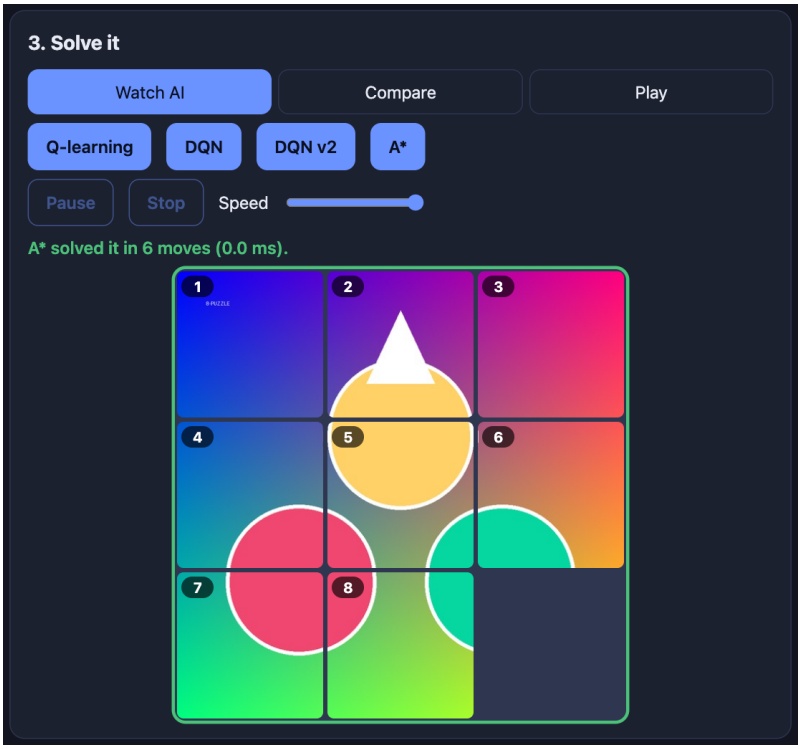


Figure 8.4: Watch AI mode: A* animating a solution

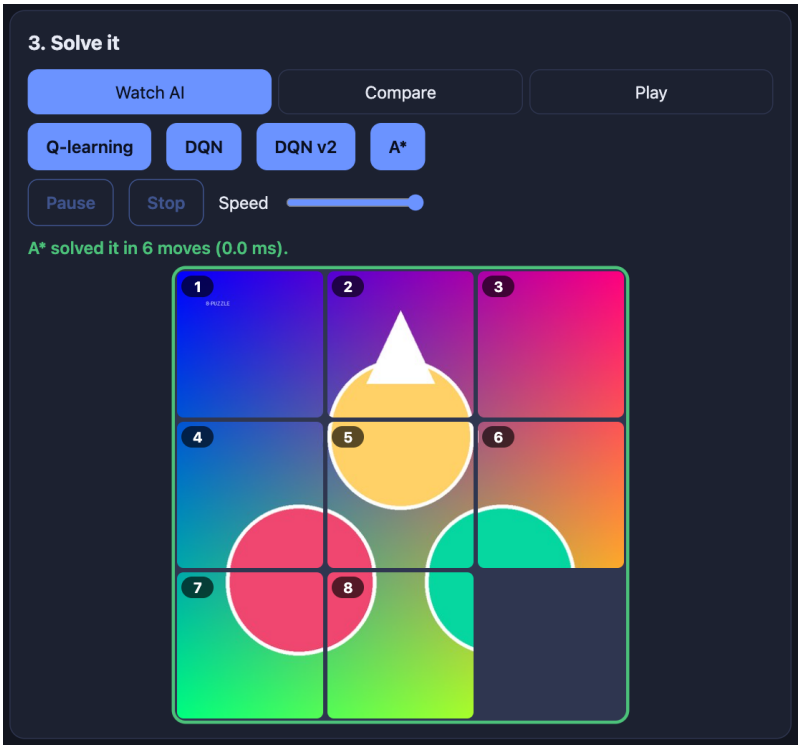


Figure 8.5: A* has solved the puzzle (optimal moves and time are shown)

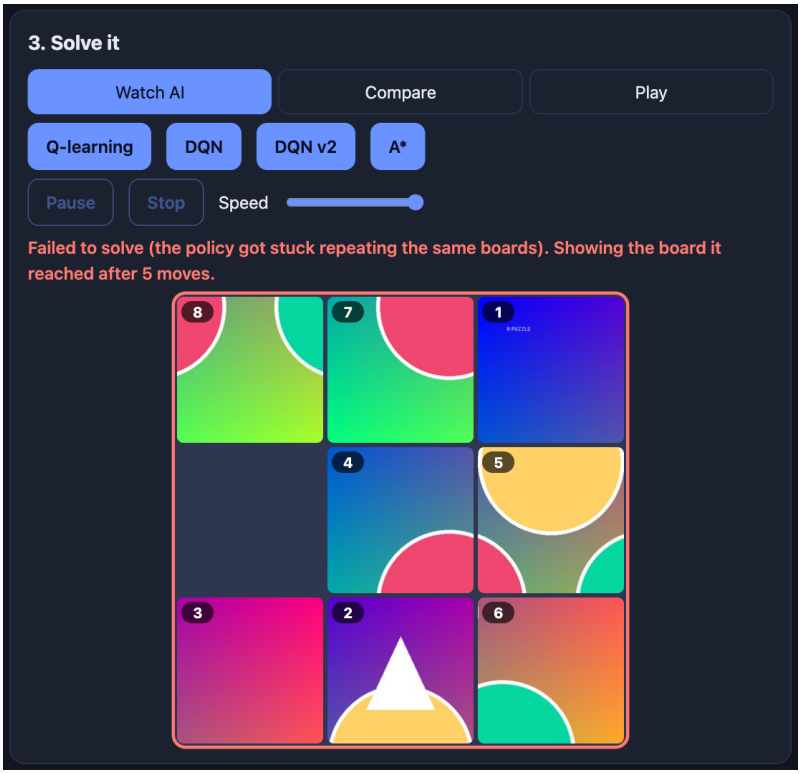


Figure 8.6: A failure is shown honestly: DQN v1 looped, and the board it reached is displayed

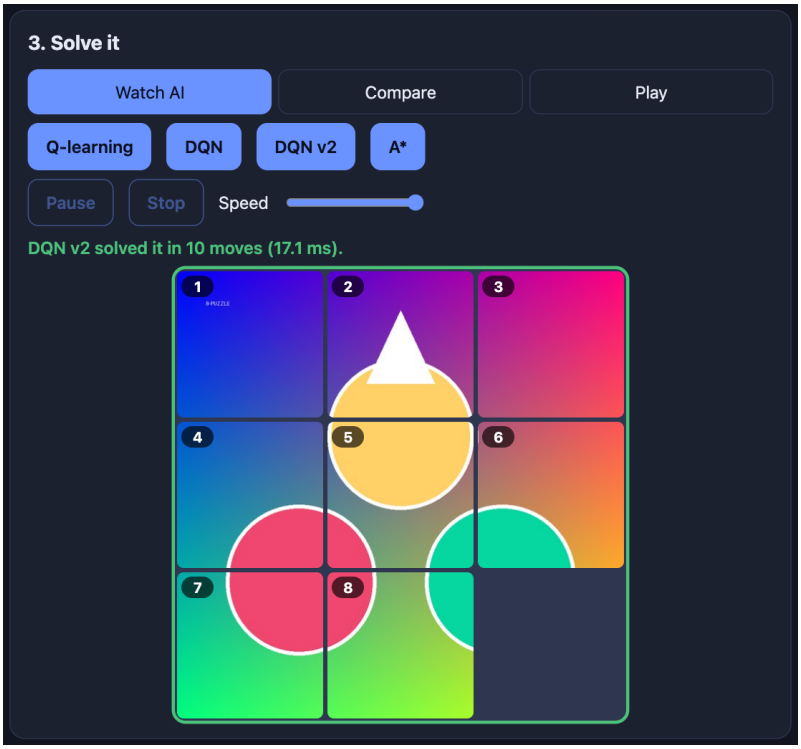


Figure 8.7: DQN v2 solving a board

Compare

The compare mode runs all four solvers on the same board and animates four small boards together. A green frame means solved and a red frame means failed. The table below them lists whether each solver solved the board, the number of moves, the time and a note (Figure 8.8).

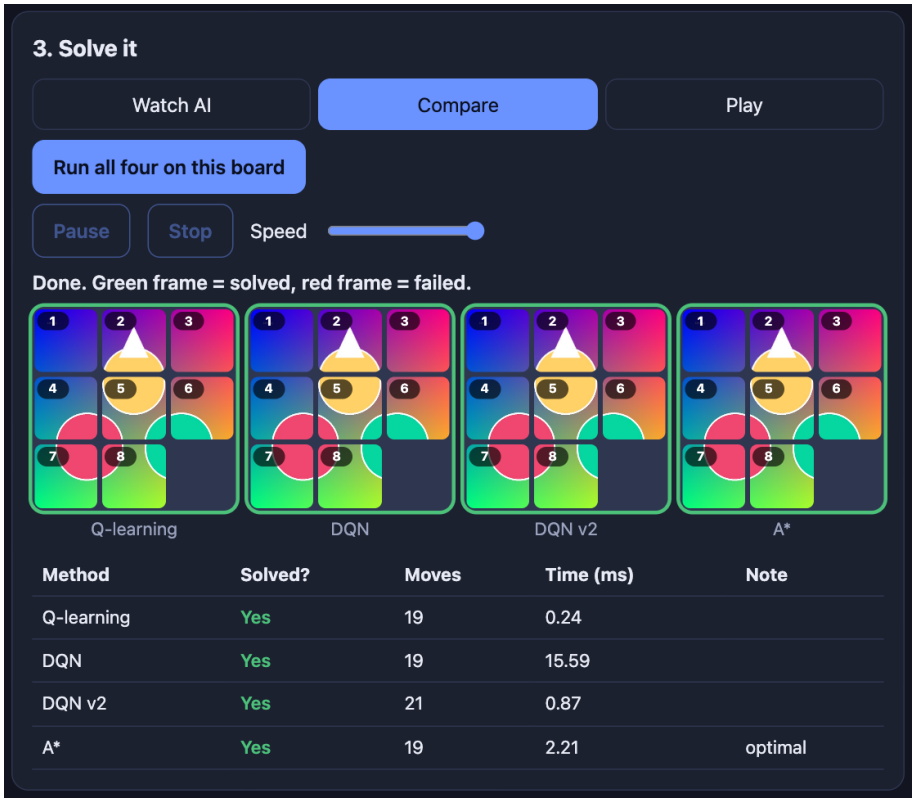


Figure 8.8: Compare mode: four solvers on the same depth-25 board

Play

In play mode the user slides tiles by clicking a tile next to the gap or with the arrow keys. A move counter and a timer run, a success message appears when the puzzle is solved and the user's move count is compared with the optimal count from A*. The Hint button highlights the tile that A* would move next (Figure 8.9), and Reset returns to the scrambled board.

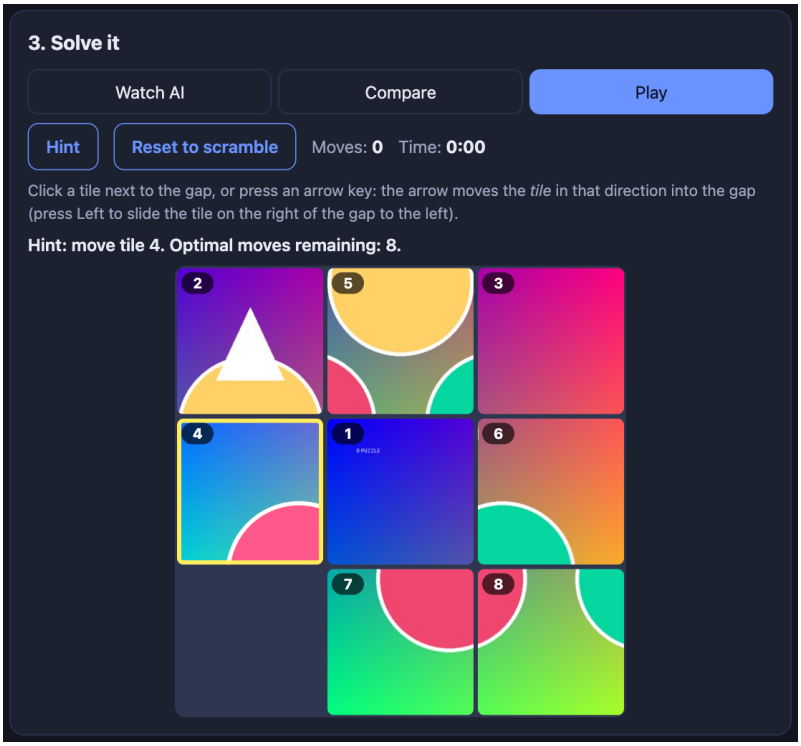


Figure 8.9: Play mode with a hint (highlighted tile)

8.5 Training Results Panel and Mobile Layout

The bottom of the page shows the overall results from comparison.csv and some of the plots of Chapter 9 (Figure 8.10). The layout adapts to a phone-sized screen (Figure 8.11).

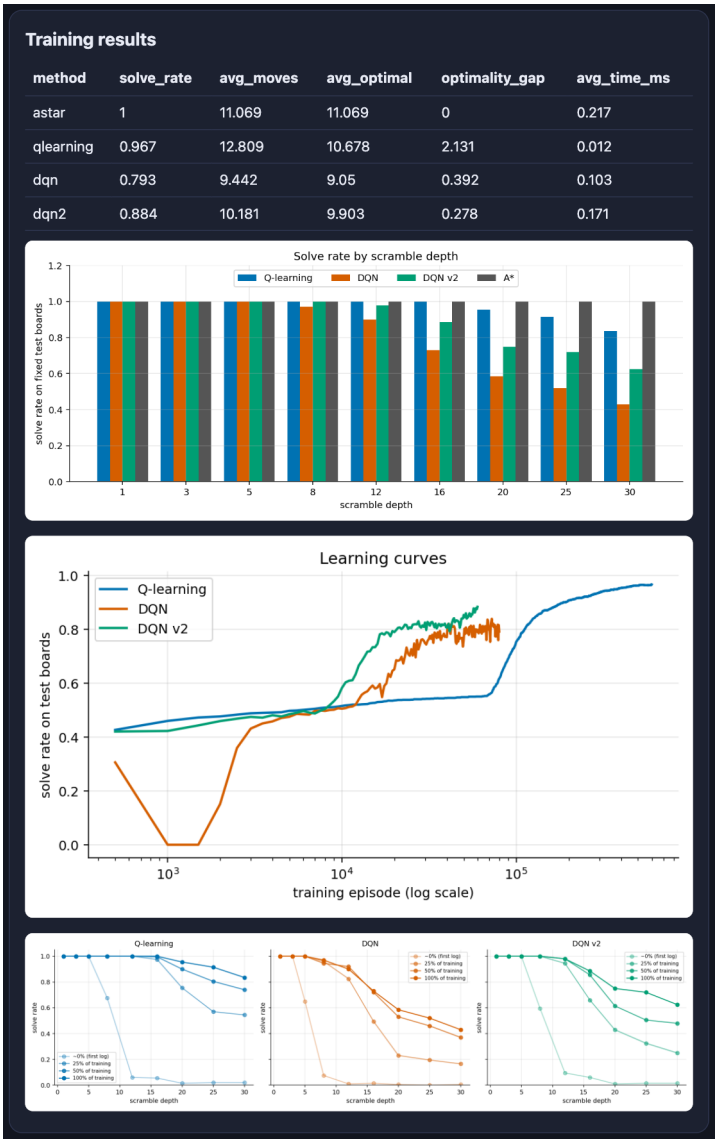


Figure 8.10: Training-results panel inside the web page

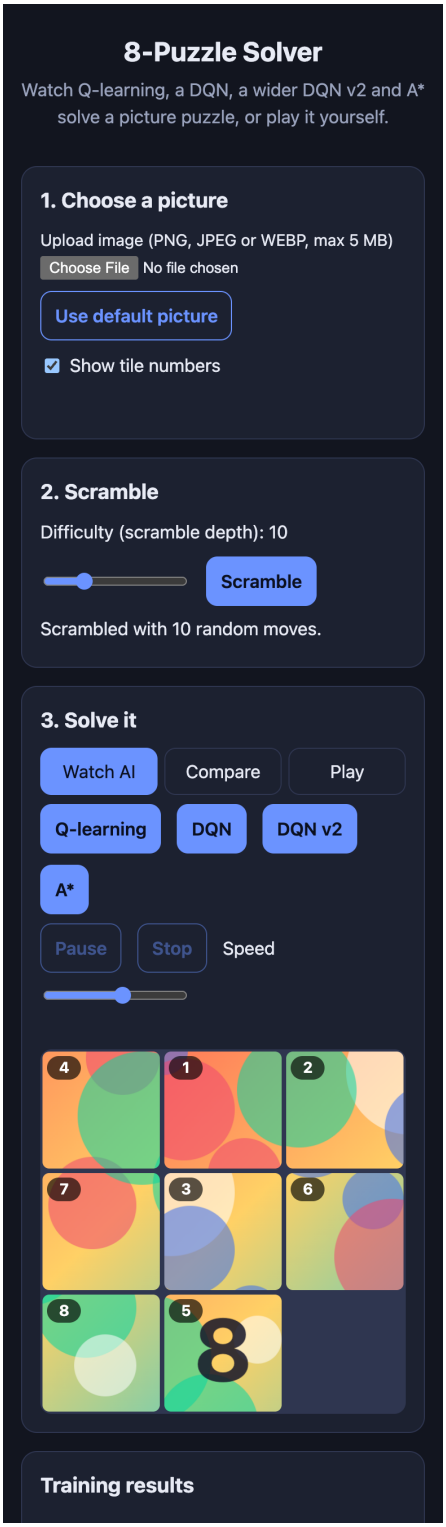


Figure 8.11: The page on a phone-sized screen (390 px wide)

8.6 Technologies Used

Table 8.2: Technologies used

Part	Technology
Language	Python 3.9 (code runs on 3.9 or newer)
Neural networks	PyTorch (CPU only)
Numerical work and data	NumPy, pandas
Plots and GIFs	Matplotlib, Pillow, ImageIO
Progress bar	tqdm
Web server	Flask; HTML, CSS and JavaScript (no external libraries)
Image handling in the server	Pillow, in memory only

8.7 Running the Project

All commands are run from the project folder. Install with `pip install -r requirements.txt`; train with `python -m puzzle.train_q` and `python -m puzzle.train_dqn`; evaluate with `python -m puzzle.evaluate`; create the plots with `python -m puzzle.visualize`; and start the web application with `python webapp/app.py`, which serves the page at http://127.0.0.1:5000.

Chapter 9
Results

9.1 Evaluation Metrics

All methods were run on the same 1,800 test boards with a limit of 100 moves for the learned solvers. The following measures are used.

- **Solve rate** = boards solved / boards tried.
- **Average moves** = mean solution length over the boards that the method solved.
- **Optimality gap** = mean of (moves used − A* optimal moves) over the boards the method solved; 0 means optimal.
- **Average time** = mean solve time per board over all boards, in milliseconds.

9.2 Overall Performance

Table 9.1: Overall performance on the 1,800 test boards

Method	Solve rate	Avg. moves (solved)	Optimality gap	Avg. time (ms)
A*	100.0%	11.07	0.00	0.217
Q-learning	96.7%	12.81	2.13	0.012
DQN v1	79.3%	9.44	0.39	0.103
DQN v2	**88.4%**	**10.18**	**0.28**	**0.171**

A* solves every board and is the reference. Tabular Q-learning solves 96.7% of the boards, and DQN v2 solves 88.4% compared with 79.3% for DQN v1. DQN v1 and v2 have lower “average moves” than A* only because they solve mostly the easier boards; the optimality gap, which compares with A* on the *same* boards, is the fairer measure. Q-learning is the fastest because a solve is a series of dictionary look-ups, while A* and the networks spend time on search or on matrix products. All of them are far below one millisecond per board.

9.3 Performance by Scramble Depth

Table 9.2: Solve rate by scramble depth

Depth	A*	Q-learning	DQN v1	DQN v2
1	100.0%	100.0%	100.0%	100.0%
3	100.0%	100.0%	100.0%	100.0%
5	100.0%	100.0%	100.0%	100.0%
8	100.0%	100.0%	97.0%	100.0%
12	100.0%	100.0%	90.0%	98.0%
16	100.0%	100.0%	73.0%	88.5%
20	100.0%	95.5%	58.5%	75.0%
25	100.0%	91.5%	52.0%	72.0%
30	100.0%	83.5%	43.0%	62.5%
Overall	**100.0%**	**96.7%**	**79.3%**	**88.4%**

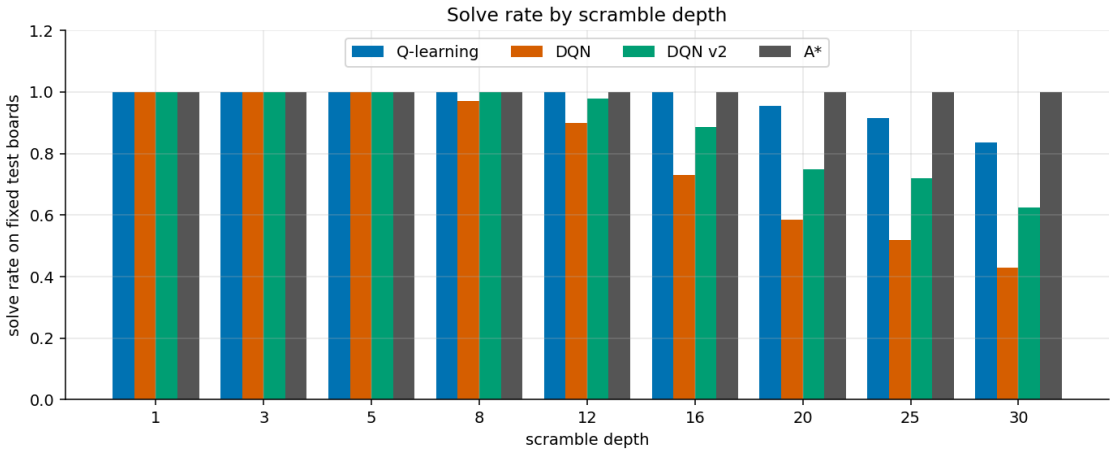


Figure 9.1: Solve rate by scramble depth for the four methods

All methods solve every board up to depth 5. From depth 8 on, the first DQN starts to fail; at depth 30 it solves only 43.0%, DQN v2 solves 62.5% and Q-learning solves 83.5%.

9.4 Solution Quality

Table 9.3: Extra moves compared with A* on the boards each method solved

Depth	Q-learning gap	DQN v1 gap	DQN v2 gap
1	0.00	0.00	0.00
3	0.00	0.00	0.00
5	0.00	0.00	0.00
8	0.00	0.00	0.00
12	0.05	0.13	0.04
16	2.48	0.62	0.30
20	4.87	1.16	0.60
25	5.77	1.29	0.92
30	7.29	2.05	1.26
Overall	**2.13**	**0.39**	**0.28**

When a network solves a board, its solution is almost optimal (an average of 0.28 extra moves for DQN v2). Q-learning solutions are longer: 2.13 extra moves on average, and about 7 extra moves at depth 30. The table agent learns a safe path to the goal but does not always find the shortest one.

9.5 Learning Curves and Reward

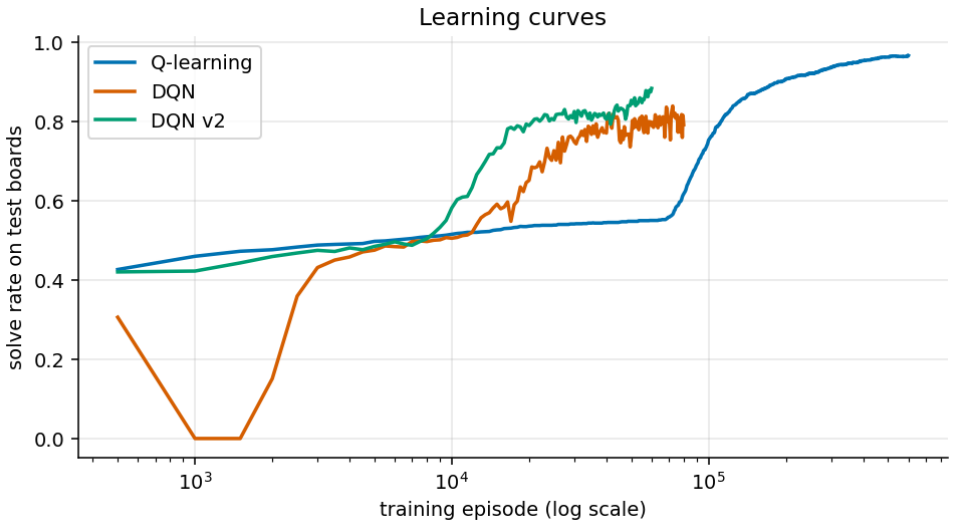


Figure 9.2: Solve rate on the test boards during training (logarithmic episode axis)

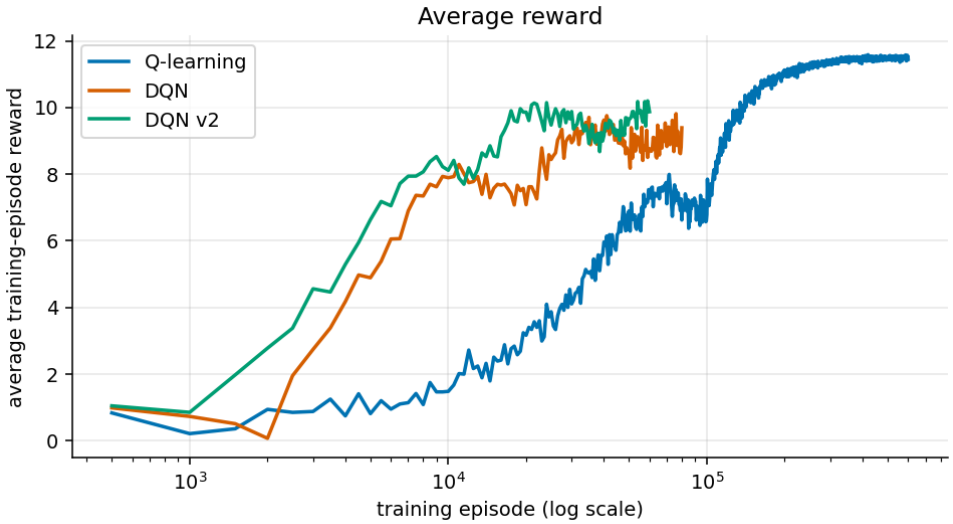


Figure 9.3: Average reward per training episode

Q-learning improves slowly for the first 10,000 episodes (its table is still small) and then rises quickly to about 97%. Both DQNs rise earlier but level off at a lower value. The curve of DQN v1 dips at the start because the early network plays almost at random. The average reward is not directly comparable between methods, because it depends on the depth the curriculum has reached.

9.6 Improvement During Training (Snapshots)

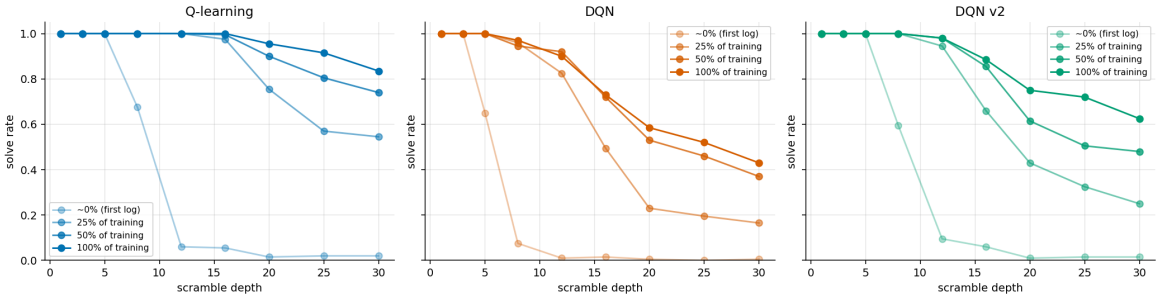


Figure 9.4: Solve rate by depth at about 0%, 25%, 50% and 100% of training

The snapshots show how the agents improve. After the first 500 episodes (the “0%” snapshot) all three fail beyond depth 8. Each later snapshot is better, and for Q-learning the gain between 25% and 100% of training is large at depths 20 to 30. For both DQNs the 50% and 100% snapshots are close, which suggests that simply training longer would not close the gap to Q-learning.

9.7 Generalization of the Q-Table

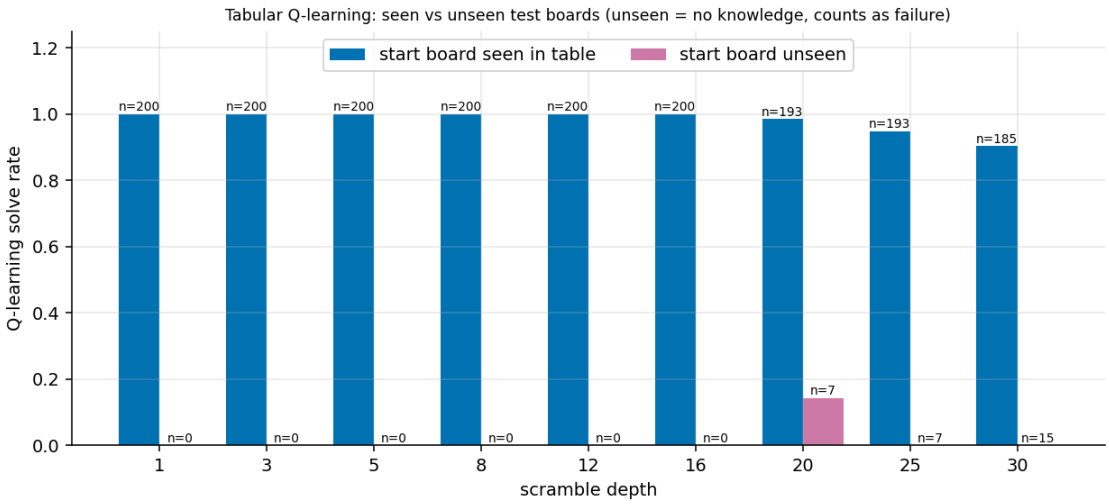


Figure 9.5: Q-learning: test boards whose start state is in the table (seen) or not (unseen)

Of the 1,800 test boards, 1771 are present in the final Q-table and 29 are not. Seen boards are solved 98.2% of the time and unseen ones only 3.4% of the time. A table has no way of generalising: a board it has never visited carries no information, and the solver reports the failure reason “unseen_state”. This is the main weakness that neural networks are meant to fix, although in this small puzzle the table covers almost the whole state space.

9.8 Failure Analysis

Table 9.4: Why the learned solvers failed (1,800 test boards)

Method	Failed boards	loop	unseen_state	step_cap
Q-learning	59	29	30	0
DQN v1	373	373	0	0
DQN v2	208	208	0	0

Almost every failure is a loop: the policy moves a tile and then moves it back, because the network (or table) rates two neighbouring boards as each other's best next board. In a separate check on the failed deep boards of DQN v1, most repeated boards were exactly this kind of two-move undo.

9.9 Example Solution

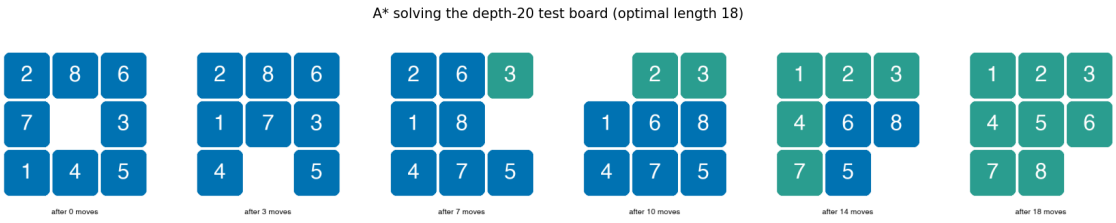


Figure 9.6: Frames of an optimal A* solution for a depth-20 test board

The project also produces GIF animations of the same hard board being solved (or not) by A*, and by Q-learning and DQN at the early snapshot and at the end of training; they are stored in the results folder.

9.10 An Extra Experiment: Starting Q-Learning from Random Boards

After the main runs we tried one more idea on Q-learning, as a separate experiment that does not change the results above. We continued the trained table for 600,000 more episodes in which every episode starts from a *uniformly random solvable board* instead of a curriculum scramble, with epsilon decreasing from 0.15 to 0.02. In about 2.5 minutes the table grew to 181,312 of the 181,440 solvable boards, and the solve rate on the same test set rose from 96.7% to 99.4% (depth 30: from 83.5% to 97.5%). This suggests that tabular Q-learning can get very close to solving the game completely when it is trained on starting boards from the whole state space.

Chapter 10
Observations

10.1 What Worked Well

- **A*** is perfect on this puzzle: 100% solved, always optimal, under one millisecond.
- **Tabular Q-learning** solved 96.7% of the test boards and 100% of boards up to depth 16. The curriculum and the shaping reward made learning possible at all, because the agent starts near the goal and moves outwards.
- **DQN v2** solves more boards than DQN v1 (88.4% against 79.3%) and its solutions are the closest to optimal (gap 0.28).
- **Honest evaluation:** fixed test boards, separate validation boards, no test boards as training starts, and failures reported instead of hidden.

10.2 What Did Not Work Well

- Both DQNs fail on deep scrambles: at depth 30 DQN v1 solves 43% and DQN v2 62.5%, against 83.5% for Q-learning.
- The first DQN training run diverged, and one later run with more updates per step also collapsed.
- n-step returns and a deeper network gave no benefit in our short experiments.

10.3 Why the DQN Does Worse than the Table

The following are our explanations; we did not isolate each of them with a separate experiment.

1. **The state space is small.** The table holds 164,443 boards exactly. The network has only 27,524 (v1) or 87,812 (v2) weights and has to compress the values of about 181,000 boards into them, so neighbouring boards get slightly wrong values.
2. **Errors add up along a long solution.** The reward arrives only at the goal, and every one-step update takes part of its target from an estimate that is itself a little wrong. Over 20 or 30 moves these errors accumulate.
3. **Small value errors cause loops.** A greedy policy only needs one wrong comparison between two neighbouring boards to move back and forth for ever. Our failure analysis shows that loops are almost the only failure type.
4. **Training is less stable.** Deep Q-learning mixes function approximation, bootstrapping and off-policy data, which can diverge; this is why soft target updates, Double DQN and a decaying learning rate helped.

10.4 Effect of the Design Decisions

Table 10.1: Summary of design decisions

Decision	Effect observed
Curriculum	Allowed learning to start; without it, deep boards would almost never reach the goal reward
Manhattan shaping reward	Gives a hint at every move; potential-based, so the optimal policy is unchanged
Lower learning rate (1e-3 → 3e-4)	Removed the divergence of the first DQN run
Soft target update + Double DQN + lr decay	Main reasons DQN v2 beats DQN v1 (79.3% → 88.4%)
Wider hidden layers (128 → 256)	Chosen on validation boards (82.9% against 80.4%); a small gain, not proven to be significant
n-step returns, deeper network	No improvement in our runs; left switched off
More gradient steps per environment step	Unstable (one run collapsed)
Starting boards from the whole state space (Q-learning)	96.7% → 99.4% in an extra experiment

10.5 Limits of the Results

- One training run (seed 0) per method; DQN results in particular can change from run to run.
- Most of the DQN tuning looked at the test-set log, so the DQN test numbers are slightly optimistic. Only the final choice between the last two v2 runs, and the checkpoint inside the run, used the separate validation boards.
- At shallow depths the test set contains few distinct boards, so the results there say little about learning.

10.6 Summary

Table 10.2: Summary of the three approaches

Method	Solve rate	Remark
A*	100.0%	Optimal, exact, the reference
Q-learning	96.7%	Very good; solutions a few moves longer; cannot handle unseen boards
DQN v2	88.4%	Near-optimal when it succeeds; loops on deep boards
DQN v1	79.3%	Weak on deep boards; unstable to train

Chapter 11
Conclusion and Future Scope

11.1 Conclusion

In this project we built an 8-puzzle environment that only produces solvable boards, an optimal A* baseline, a tabular Q-learning agent and two Deep Q-Networks, together with a fixed test set, plots, GIFs and a Flask web application. All training runs on a laptop CPU, is seeded, and can be stopped and resumed.

On 1,800 test boards A* solved 100%, tabular Q-learning 96.7%, DQN v2 88.4% and DQN v1 79.3%. The learned methods are almost optimal on the boards they solve but fail by looping on deep scrambles, and the web application shows these failures honestly. The experiments taught us how a curriculum, a shaping reward, a target network, Double DQN, soft target updates and a learning-rate schedule affect the stability and the quality of reinforcement learning. A final experiment showed that tabular Q-learning reaches 99.4% when it is trained on starting boards from the whole state space, so near-complete solving of this small puzzle is possible.

11.2 Future Scope

Table 11.1: Future scope

Area	Improvement
Q-learning	Train from uniformly random solvable start boards (a --uniform-start option) and decay the learning rate to try to reach 100%
DQN	Uniform start boards, prioritized experience replay [15], and a longer run with a decaying learning rate
Loop avoidance	Forbid undoing the previous move while solving; it would remove the most common failure and should be tested
Stronger agents	Rainbow-style combinations [17] or learning from A* solutions as extra supervision
Several runs	Repeat with different seeds and report mean and spread
Larger puzzles	Apply to the 15-puzzle, where a table cannot fit in memory
Explanation	Show the Q-values of the four moves in the web page
Deployment	Host the web application on a cloud service with a production server

References

[1] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.

[3] P. E. Hart, N. J. Nilsson and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] H. van Hasselt, A. Guez and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI*, 2016.

[5] A. Y. Ng, D. Harada and S. Russell, “Policy invariance under reward transformations: theory and application to reward shaping,” in *Proc. ICML*, 1999, pp. 278–287.

[6] Y. Bengio, J. Louradour, R. Collobert and J. Weston, “Curriculum learning,” in *Proc. ICML*, 2009, pp. 41–48.

[7] R. E. Korf, “Depth-first iterative-deepening: an optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[8] F. Agostinelli, S. McAleer, A. Shmakov and P. Baldi, “Solving the Rubik's cube with deep reinforcement learning and search,” *Nature Machine Intelligence*, vol. 1, pp. 356–363, 2019.

[9] W. W. Johnson and W. E. Story, “Notes on the ‘15’ puzzle,” *American Journal of Mathematics*, vol. 2, no. 4, pp. 397–404, 1879.

[10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[11] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. ICLR*, 2015.

[12] L.-J. Lin, “Self-improving reactive agents based on reinforcement learning, planning and teaching,” *Machine Learning*, vol. 8, pp. 293–321, 1992.

[13] A. Paszke et al., “PyTorch: an imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, 2019.

[14] Pallets Projects, “Flask Documentation,” https://flask.palletsprojects.com

[15] T. Schaul, J. Quan, I. Antonoglou and D. Silver, “Prioritized experience replay,” in *Proc. ICLR*, 2016.

[16] A. Reinefeld, “Complete solution of the eight-puzzle and the benefit of node ordering in IDA*,” in *Proc. IJCAI*, 1993, pp. 248–253.

[17] M. Hessel et al., “Rainbow: combining improvements in deep reinforcement learning,” in *Proc. AAAI*, 2018.

Appendix A Project Structure

```
mini project/
|-- requirements.txt
|-- README.md
|-- puzzle/
|   |-- env.py            # state, moves, scramble, parity, greedy solver
|   |-- astar.py          # A* baseline (Manhattan distance)
|   |-- common.py         # reward, curriculum, test/validation sets, logging
|   |-- qlearn.py         # tabular Q-learning
|   |-- dqn.py            # DQN v1 / v2 (replay buffer, target network)
|   |-- train_q.py        # CLI: python -m puzzle.train_q
|   |-- train_dqn.py      # CLI: python -m puzzle.train_dqn
|   |-- evaluate.py       # test set and comparison
|   |-- visualize.py      # plots and GIFs
|   |-- logs/             # q_log.csv, dqn_log.csv, dqn2_log.csv
|   |-- checkpoints/      # latest + snapshots (0/25/50/100 %) + best
|   `-- results/          # test_set.json, comparison.csv/.md, PNGs, gifs/
|-- tests/                # unit tests (solvable boards, A* correctness)
`-- webapp/
    |-- app.py            # Flask server
    |-- templates/index.html
    `-- static/ (js, css)
```

Listing A.1: Project folder structure

Disclaimer

This is a student project made for educational purposes. The learned solvers (Q-learning and DQN) do not guarantee a solution; only A* is guaranteed to find an optimal one. Results are from a single training run and should be read with the limits described in Chapter 10.